
Climate Monitoring & Insights Dashboard

Infosys Springboard Project Report

Name	Shaik Fathima Saniya
Program	Infosys Springboard
Mentor	Shanmukha Priya
Tools	Python • Plotly • Streamlit
Dataset	Global Weather Repository (Kaggle)

Acknowledgements

This project was completed as part of the **Infosys Springboard** program. Special thanks to mentor **Shanmukha Priya** for their continued guidance, constructive feedback, and encouragement at every stage of this project. Their insights into real-world data analytics practices — from handling data quality issues to designing effective visualizations — significantly shaped both the direction and the quality of this work.

Abstract

This report presents the design, implementation, and analytical findings of the **Climate Monitoring & Insights Dashboard**, a comprehensive interactive web-based climate analytics platform built in Python and Streamlit as part of the Infosys Springboard program.

The platform addresses a critical gap in climate data accessibility: global weather data is widely available but rarely presented in a form that is simultaneously statistically rigorous and interpretable by non-technical decision-makers. This dashboard bridges that gap by consolidating multi-variable global weather data — including temperature, precipitation, wind speed, humidity, UV index, and air quality — into nine purpose-built analytical modules, each answering a distinct and actionable climate question.

The system is built on a three-stage pipeline. The first stage implements a data preprocessing pipeline covering raw data inspection, missing value imputation, duplicate removal, column reduction, unit standardization from imperial to metric, MinMax normalization, and temporal feature engineering. The second stage performs comprehensive exploratory data analysis including statistical summarization, Pearson correlation analysis, seasonal decomposition, extreme event detection, and five global choropleth visualizations. The third stage delivers an interactive nine-module Streamlit dashboard with globally linked filters, dynamic 90th-percentile extreme event thresholding, and a composite multi-variable climate risk scoring formula.

The system identified 19,411 extreme weather events distributed nearly equally across heatwave (33.1%), heavy rain (33.0%), and high wind (33.9%) categories. Gulf states and arid Central Asian countries dominate the composite risk rankings.

Keywords: Climate Analytics, Streamlit, Python, Data Preprocessing, Exploratory Data Analysis, Extreme Event Detection, Choropleth Maps, Seasonal Analysis, Pearson Correlation, Risk Scoring.

Contents

Acknowledgements	1
Abstract	2
List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Project Background	7
1.2 Objectives	7
1.3 Dataset Description	8
1.4 Three-Stage Pipeline	8
1.5 Technology Stack	9
2 Proposed System	10
2.1 System Overview	10
2.2 System Architecture	10
2.2.1 Stage 1: Data Preprocessing Pipeline	10
2.2.2 Stage 2: Core Analysis	11
2.2.3 Stage 3: Interactive Streamlit Dashboard	11
2.3 Dashboard Design	12
2.4 Key Technical Innovations	12
2.4.1 Dynamic Percentile Thresholding	12
2.4.2 Composite Climate Risk Score	13
2.4.3 Globally Linked Filter Architecture	13
3 Data Preprocessing	14
3.1 Overview	14
3.2 Step 1: Data Loading and Initial Inspection	14
3.3 Step 2: Missing Value and Anomaly Analysis	15
3.4 Step 3: Data Cleaning	15
3.5 Step 4: Irrelevant Column Removal	16
3.6 Step 5: Unit Conversion and MinMax Normalization	16
3.7 Step 6: Temporal Feature Engineering	17
3.8 Step 7: Export Cleaned Dataset	18
3.9 Preprocessing Summary	18
4 Core Exploratory Data Analysis	19
4.1 Overview	19
4.2 Statistical Summary	19
4.3 Correlation Analysis	19
4.3.1 Full Dataset Correlation Matrix	19

4.3.2	Monthly Aggregated Correlation	20
4.4	Seasonal Temperature Analysis	20
4.5	Seasonal Rainfall Analysis	21
4.6	Yearly Temperature Trend	22
4.7	Extreme Event Detection	23
4.8	Country-Level Aggregation	24
4.9	Variable Distribution Analysis	24
4.10	Five Global Choropleth Maps	25
5	Interactive Dashboard	27
5.1	Overview	27
5.2	Application Setup and Column Detection	27
5.3	Global Sidebar Filters	27
5.4	Module 1: Global Climate Executive Overview	28
5.5	Module 2: Temperature Intelligence	29
5.6	Module 3: Rainfall and Wind Intelligence	30
5.7	Module 4: Extreme Weather Events Monitor	31
5.8	Module 5: Regional Comparison	32
5.9	Module 6: Climate Risk Intelligence	33
5.10	Module 7: Climate Heatmap	34
5.11	Seasonal Analysis	34
5.12	Climate Correlation	35
6	Results	37
6.1	Data Preprocessing Results	37
6.2	Core Analysis Results	37
6.2.1	Statistical Summary	37
6.2.2	Correlation Results	38
6.2.3	Seasonal Pattern Results	38
6.3	Dashboard Results	38
6.3.1	Extreme Weather Events	38
6.3.2	Climate Risk Rankings	39
6.3.3	Dashboard Features Delivered	39
7	Conclusion	40
7.1	Summary	40
7.2	Key Insights	40
7.3	Limitations and Future Work	41
A	Data Dictionary	42
B	Deployment Instructions	43

List of Figures

2.1	Proposed system Executive dashboard overview showing the final delivered interface	10
4.1	Correlation Heatmap	20
4.2	Seasonal Temperature Analysis	21
4.3	Seasonal Rainfall Analysis	22
4.4	Yearly Temperature Trend	23
4.5	Global Choropleth Map(country)	26
5.1	Global Climate Executive Overview	29
5.2	Temperature Intelligence	30
5.3	Rainfall & Wind Intelligence	31
5.4	Extreme Weather Events Monitor	32
5.5	Country Comparison	33
5.6	Climate Risk Intelligence	34
5.7	Climate Heatmap	34
5.8	Seasonal Climate Analysis	35
5.9	Climate Correlation	36

List of Tables

1.1	Dataset description and scope	8
1.2	Three-stage project pipeline	8
1.3	Technology stack	9
2.1	Nine dashboard modules and the question each answers	12
3.1	Preprocessing steps and the problem each solves	14
3.2	Preprocessing complete deliverable summary	18
6.1	Data preprocessing outcomes	37
6.2	Key statistical results from core analysis	37
6.3	Key Pearson correlation findings	38
6.4	Extreme weather event breakdown from Module 4	38
6.5	Top 10 countries by composite climate risk score	39
6.6	All dashboard features delivered	39
7.1	Limitations and proposed future enhancements	41
A.1	Data dictionary: retained and engineered columns	42

1. Introduction

1.1 Project Background

Climate change is one of the most pressing challenges of the modern era. Global temperatures are rising steadily, seasonal rainfall patterns are becoming unpredictable, and extreme weather events such as heatwaves, floods, and high-wind storms are increasing in both frequency and severity. Governments, policymakers, agricultural planners, insurance agencies, and disaster management authorities all need timely, reliable, and interpretable climate data to protect lives, livelihoods, and critical infrastructure.

Despite the growing availability of global weather datasets, a persistent gap exists between raw data availability and actionable analytical insight. Most existing tools either require deep technical expertise to operate, such as command-line data science environments, or sacrifice analytical depth in favour of simplicity, such as basic weather reporting apps. Neither approach meets the needs of an analyst who requires both statistical rigour and a presentation that non-technical stakeholders can understand and act on.

This project was built to close that gap. The **Climate Monitoring & Insights Dashboard** transforms a raw global weather dataset into a fully interactive analytics platform. The result is a web-based dashboard that any user — technical or non-technical — can navigate, filter, and interpret to derive meaningful, evidence-based climate insights.

1.2 Objectives

The project was designed to achieve the following objectives:

- **Data Preprocessing:** Clean, standardize, normalize, and enrich the raw weather dataset so that it is fully analysis-ready with zero missing values, zero duplicates, consistent metric units, and properly extracted temporal features. Every downstream result depends on the integrity of this output.
- **Core Analysis:** Explore the cleaned dataset through statistical summarization, Pearson correlation analysis, seasonal decomposition, extreme event detection, country-level aggregation and ranking, and global choropleth visualizations. The findings directly inform the design of each dashboard module.
- **Interactive Dashboard:** Deliver a nine-module Streamlit dashboard with a globally linked filter system, dynamic 90th-percentile extreme event thresholding, a composite multi-variable climate risk scoring formula, and interactive Plotly visualizations covering temperature, precipitation, wind, seasonality, risk, and environmental correlations.

1.3 Dataset Description

The dataset used in this project is the **GlobalWeatherRepository**, a comprehensive multi-country weather dataset sourced from Kaggle, covering daily climate observations from 2024 to 2026. Each record represents one day of weather observations for one country and includes a wide range of atmospheric and environmental measurements.

Parameter	Details
Geographic Coverage	Multiple countries worldwide
Time Period	2024 – 2026
Key Variables Retained	Temperature (Celsius), Precipitation (mm), Wind Speed (kph), Humidity (%), Atmospheric Pressure (mb), UV Index, Feels-Like Temperature (Celsius), Air Quality (US EPA Index)
Columns Removed	Imperial unit duplicates, lunar data, individual air quality pollutants, timezone, wind direction (22 columns total)
Normalization	All numeric columns normalized to [0, 1] using MinMaxScaler

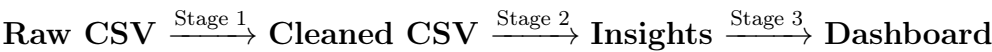
Table 1.1: Dataset description and scope

1.4 Three-Stage Pipeline

The project is organized as a sequential three-stage pipeline where each stage builds on the output of the previous one.

Stage	Name	Input / Output
1	Data Preprocessing	Input: Raw CSV Output: Cleaned, normalized CSV
2	Core Analysis	Input: Cleaned CSV Output: Statistical findings, seasonal insights, correlation structure, geographic maps
3	Interactive Dashboard	Input: Cleaned CSV Output: Nine-module interactive web dashboard

Table 1.2: Three-stage project pipeline



1.5 Technology Stack

Stage	Library	Purpose
1, 2, 3	Pandas, NumPy	Data loading, filtering, groupby, pivot tables, date parsing
1	Scikit-learn	MinMaxScaler normalization
2	Matplotlib	Static EDA charts
2	Seaborn	Correlation heatmaps
2, 3	Plotly Express	Interactive charts and choropleth maps
3	Streamlit	Web dashboard, widgets, caching, layout

Table 1.3: Technology stack

2. Proposed System

2.1 System Overview

The proposed system is the **Climate Monitoring & Insights Dashboard** — an end-to-end data analytics platform that ingests raw global weather data, processes it through a structured pipeline, and delivers the results through an interactive nine-module web interface. The system serves two user groups simultaneously: technical analysts who require statistical depth, and non-technical decision-makers who require interpretable outputs they can act on immediately.

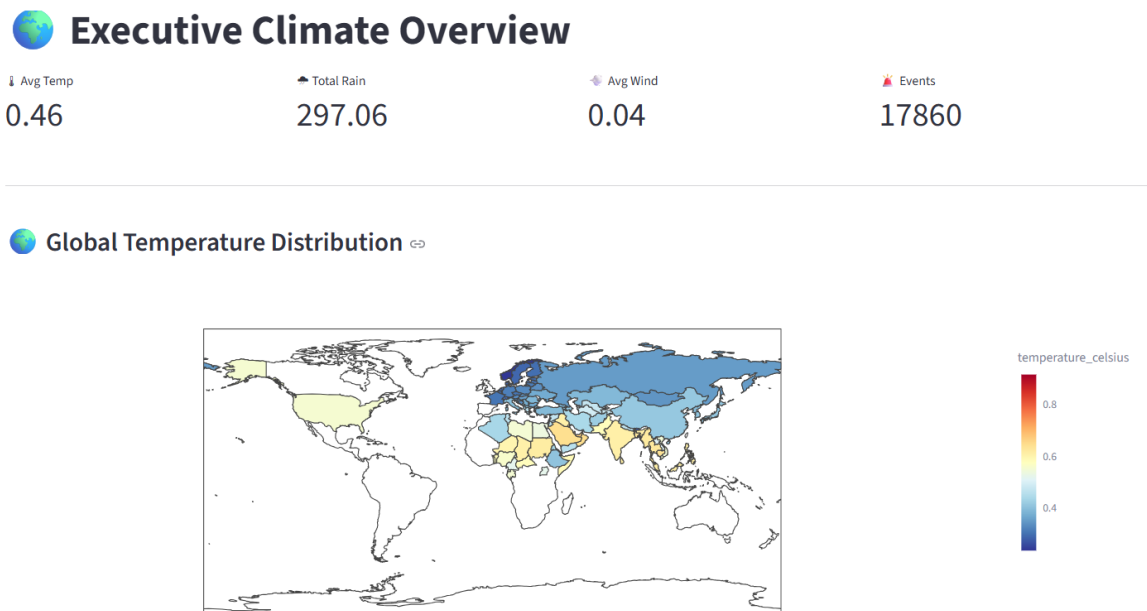


Figure 2.1: Proposed system Executive dashboard overview showing the final delivered interface

2.2 System Architecture

2.2.1 Stage 1: Data Preprocessing Pipeline

The preprocessing stage is a seven-step sequential pipeline that transforms the raw CSV into a clean, normalized, analysis-ready file. It handles all data quality issues — duplicates, missing values, unit inconsistencies, redundant columns, and missing temporal features — before any analysis is performed.

The output file contains:

- Zero missing values and zero duplicate rows

- All measurements in consistent metric units (Celsius, kph)
- All numeric features normalized to $[0, 1]$ using MinMaxScaler
- Year and Month temporal features extracted for time-series analysis
- Dimensionality reduced by 22 dropped redundant columns

2.2.2 Stage 2: Core Analysis

The analysis stage takes the cleaned CSV and performs comprehensive exploratory data analysis. It produces seven categories of output: statistical summaries, correlation matrices, seasonal temperature patterns, seasonal rainfall patterns, year-on-year temperature trends, extreme event detection by country, and five interactive global choropleth maps.

2.2.3 Stage 3: Interactive Streamlit Dashboard

The dashboard loads the cleaned CSV and renders nine modules through an interactive filter-driven interface. Three core architectural principles govern it:

- **Global filter consistency:** A country multi-select and a year range slider overwrite the shared dataframe before any module renders, ensuring all nine modules always reflect the same data scope.
- **Performance through caching:** The `@st.cache_data` decorator loads the CSV once per session, eliminating the overhead of re-reading from disk on every interaction.
- **Resilience through dynamic column detection:** Key columns are identified by substring matching rather than hardcoded names, making the system robust to schema changes.

2.3 Dashboard Design

Module	Name	Question Answered
1	Executive Overview	What is the global climate situation across all selected countries?
2	Temperature Intelligence	How are temperatures distributed, trending, and varying?
3	Rainfall & Wind	How do precipitation and wind behave and relate to each other?
4	Extreme Events Monitor	What types of extreme climate events occurred, and when?
5	Country Comparison	How do two specific countries compare in temperature?
6	Climate Risk Intelligence	Which countries face the greatest compound climate risk?
7	Climate Heatmap	What do seasonal patterns look like across years simultaneously?
8	Seasonal Analysis	When do temperature and rainfall peak across the year?
9	Climate Correlation	Which climate variables are statistically related?

Table 2.1: Nine dashboard modules and the question each answers

2.4 Key Technical Innovations

2.4.1 Dynamic Percentile Thresholding

The most technically significant innovation is the dynamic 90th-percentile threshold for extreme event detection in Module 4. Fixed absolute thresholds (e.g., temperature above 40°C) are statistically biased — 40°C is an ordinary summer day in Saudi Arabia but a catastrophic extreme in Norway. The proposed system instead computes the threshold dynamically within the filtered dataset:

$$T_{extreme} = Q_{0.90}(X_{filtered}) \quad (2.1)$$

When Gulf states are selected, the threshold calibrates to the hot-arid baseline. When Nordic countries are selected, it recalibrates to the cooler baseline. The definition of extreme is always relative to the actual data in view.

2.4.2 Composite Climate Risk Score

Module 6 synthesizes three independent stress variables into a single comparable index per country:

$$\text{risk_score}_i = \frac{T_i}{T_{max}} + \frac{R_i}{R_{max}} + \frac{W_i}{W_{max}} \quad (2.2)$$

Equal weighting ensures no single variable dominates. Normalization by maximum ensures all three terms contribute on the same $[0, 1]$ scale. Countries stressed by heat, rain, and wind simultaneously rank higher than countries extreme in only one dimension.

2.4.3 Globally Linked Filter Architecture

Both sidebar filters are applied to the shared dataframe before any module renders. This guarantees that all nine modules are always analytically consistent — changing the country selection in Module 1 is automatically reflected in Module 9 without any additional code.

3. Data Preprocessing

3.1 Overview

Raw weather data is never analysis-ready. It contains duplicate records, missing values, columns in inconsistent unit systems, irrelevant metadata, and no temporal features. This chapter implements a seven-step pipeline that addresses all of these problems and produces the single cleaned CSV that powers all downstream stages.

Step	Operation	Problem Solved
1	Data Loading & Inspection	Establish baseline understanding of structure
2	Missing Value Analysis	Quantify quality problems before cleaning
3	Cleaning	Duplicates, nulls, invalid values
4	Column Reduction	22 redundant or irrelevant columns dropped
5	Unit Conversion	All measurements standardized to metric
5	Normalization	All numerics scaled to [0, 1]
6	Feature Engineering	Year and Month extracted from date
7	Export	Cleaned CSV saved for downstream use

Table 3.1: Preprocessing steps and the problem each solves

3.2 Step 1: Data Loading and Initial Inspection

The dataset is loaded and a complete structural profile is printed before any modifications are made. The `df.describe()` call reveals the count, mean, standard deviation, min, quartiles, and max of every numeric column, providing the first indication of outliers or invalid ranges.

```
1 import pandas as pd
2
3 df = pd.read_csv("/content/GlobalWeatherRepository.csv")
4 print("Dataset Shape:", df.shape)
5 print("\nColumns:\n", df.columns)
6 print("\nData Types:\n", df.dtypes)
7 print("\nSample Data:\n", df.head())
8 print("\nNumerical Summary:\n", df.describe())
9 print("\nCategorical Summary:\n", df.describe(include='object',
    ))
```

Listing 3.1: Step 1 — Loading raw data and printing full inspection profile

3.3 Step 2: Missing Value and Anomaly Analysis

All quality problems are measured before any modification. Missing values are counted per column as both an absolute number and as a percentage of total records. Duplicate row counts are computed. Numeric column ranges are checked for physically impossible values. The date column is parsed to confirm temporal coverage of 2024–2026.

```
1 print("\nMissing Values Per Column:\n", df.isnull().sum())
2 missing_percent = (df.isnull().sum() / len(df)) * 100
3 print("\nMissing Value Percentage:\n", missing_percent)
4 print("\nDuplicate Rows:", df.duplicated().sum())
5
6 for col in df.select_dtypes(include=['int64', 'float64']).
   columns:
7     print(f"\nChecking anomalies in {col}")
8     print("Min:", df[col].min(), "Max:", df[col].max())
9
10 for col in df.columns:
11     if "date" in col.lower():
12         df[col] = pd.to_datetime(df[col], errors='coerce')
13         print("\nData Coverage:")
14         print("Start Date:", df[col].min())
15         print("End Date:", df[col].max())
```

Listing 3.2: Step 2 — Measuring missing values, duplicates, and range anomalies

3.4 Step 3: Data Cleaning

Duplicate rows are removed. Columns with more than 40% missing values are dropped entirely — imputing a majority-missing column would introduce more noise than signal. Remaining numeric nulls are imputed with the column *median*, chosen over mean because the median is robust to outliers characteristic of climate data. Categorical nulls are imputed with the column *mode*. Rows with negative numeric values are removed as physically invalid.

```
1 df = df.drop_duplicates()
2 threshold = 0.4 * len(df)
3 df = df.dropna(thresh=threshold, axis=1)
4 numeric_cols = df.select_dtypes(include=['int64', 'float64',
5     ]).columns
6 categorical_cols = df.select_dtypes(include=['object']).
   columns
7 df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].
   median())
8 for col in categorical_cols:
9     df[col] = df[col].fillna(df[col].mode()[0])
10 for col in numeric_cols:
11     df = df[df[col] >= 0]
12 print("Missing Values After Cleaning:\n", df.isnull().sum())
```

Listing 3.3: Step 3 — Duplicate removal, null imputation, negative filtering

3.5 Step 4: Irrelevant Column Removal

Twenty-two columns are dropped: Fahrenheit temperature duplicates, mph wind speed duplicates, imperial pressure and precipitation duplicates, lunar data (moonrise, moonset, moon phase, moon illumination), timezone and last-updated metadata, wind degree and direction, and seven individual air quality pollutant columns (replaced by the single composite US EPA index).

```
1 unwanted_columns = [  
2     'temperature_fahrenheit', 'wind_mph', 'pressure_in',  
3     'precip_in', 'feels_like_fahrenheit', 'visibility_miles',  
4     'last_updated', 'gust_mph', 'last_updated_epoch',  
5     'wind_degree', 'timezone', 'moonrise', 'moonset',  
6     'moon_phase', 'moon_illumination', 'wind_direction',  
7     'air_quality_Carbon_Monoxide', 'air_quality_Ozone',  
8     'air_quality_Nitrogen_dioxide', '  
9         air_quality_Sulphur_dioxide',  
10    'air_quality_PM2.5', 'air_quality_PM10',  
11    'air_quality_gb-defra-index'  
12 ]  
13 df.drop(columns=[col for col in unwanted_columns if col in df.  
14     columns],  
15     inplace=True)  
16 print("\nColumns remaining after removal:")  
17 print(df.columns)
```

Listing 3.4: Step 4 — Removing 22 redundant and irrelevant columns

3.6 Step 5: Unit Conversion and MinMax Normalization

Temperature columns are converted from Fahrenheit to Celsius, and wind speed from mph to kph. All numeric columns are then normalized to $[0, 1]$ using `MinMaxScaler`. Normalization is critical for the composite risk score in Module 6 — without it, variables with larger absolute ranges would dominate. `MinMaxScaler` is preferred over `StandardScaler` because climate variables such as precipitation are right-skewed, violating the normality assumption of `StandardScaler`.

$$T_C = \frac{(T_F - 32) \times 5}{9} \quad V_{kph} = V_{mph} \times 1.60934 \quad X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (3.1)$$

```
1 import numpy as np  
2 from sklearn.preprocessing import MinMaxScaler  
3  
4 for col in df.columns:  
5     if "temp" in col.lower():  
6         df[col] = pd.to_numeric(df[col], errors='coerce')
```

```

7         df[col] = (df[col] - 32) * 5/9
8
9     for col in df.columns:
10         if "wind" in col.lower():
11             df[col] = pd.to_numeric(df[col], errors='coerce')
12             df[col] = df[col] * 1.60934
13
14     numeric_cols = df.select_dtypes(include=['int64', 'float64']).
        columns
15     df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].
        median())
16
17     scaler = MinMaxScaler()
18     df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
19     print("Unit Conversion and Normalization completed
        successfully")

```

Listing 3.5: Step 5 — Unit conversion and MinMax normalization

3.7 Step 6: Temporal Feature Engineering

The date column is parsed into a datetime object and Year and Month features are extracted. These power every time-series view in the dashboard — trend lines, the heatmap, seasonal charts, and the year range slider.

```

1     date_col = None
2     for col in df.columns:
3         if "date" in col.lower():
4             date_col = col
5
6     if date_col:
7         df[date_col] = pd.to_datetime(df[date_col])
8         df['Year'] = df[date_col].dt.year
9         df['Month'] = df[date_col].dt.month
10        monthly_avg = df.groupby(['Year', 'Month'])[numeric_cols].
            mean().reset_index()
11        print("\nMonthly Average Data (first 5 rows):")
12        print(monthly_avg.head())
13        monthly_avg.to_csv("GlobalWeather_Monthly_Average.csv",
            index=False)

```

Listing 3.6: Step 6 — Date parsing and Year/Month feature extraction

```

1 if 'date' in df.columns:
2     df['date'] = pd.to_datetime(df['date'])
3     df['year_month'] = df['date'].dt.to_period('M')
4     monthly_avg = df.groupby('year_month')[num_cols].mean()
5     print("\nMonthly Aggregated Data:")
6     print(monthly_avg.head())

```

Listing 3.7: Step 6 continued — Year-month period for monthly grouping

3.8 Step 7: Export Cleaned Dataset

```

1 df.to_csv("GlobalWeatherRepository_Cleaned.csv", index=False)
2 print("Preprocessed Dataset Saved Successfully!")
3
4 from google.colab import files
5 files.download("GlobalWeatherRepository_Cleaned.csv")

```

Listing 3.8: Step 7 — Saving and downloading the cleaned dataset

3.9 Preprocessing Summary

Step	Operation	Outcome
1	Load & Inspect	Shape, types, date range confirmed
2	Quality Analysis	Null percentages and anomaly ranges measured
3	Cleaning	Zero nulls, duplicates, invalid values
4	Column Removal	22 columns dropped
5	Unit + Normalization	All metric; all numerics in [0, 1]
6	Feature Engineering	Year, Month, year_month added
7	Export	Cleaned CSV ready for downstream use

Table 3.2: Preprocessing complete deliverable summary

4. Core Exploratory Data Analysis

4.1 Overview

This chapter explores the cleaned dataset to understand its statistical properties, variable relationships, seasonal structures, extreme event patterns, and geographic distributions. Every analysis type performed here corresponds directly to at least one dashboard module.

4.2 Statistical Summary

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 df = pd.read_csv("GlobalWeatherRepository_Cleaned.csv")
7 print("Dataset shape:", df.shape)
8 df.head()
9
10 summary_stats = df.describe()
11 print(summary_stats)
12 summary_stats.to_csv("statistical_summary.csv")
```

Listing 4.1: Environment setup and full statistical summary

The summary confirms a mean normalized temperature of 0.46, a standard deviation of 0.19, and a full range of 0.0 to 1.0, verifying that MinMax normalization was applied correctly in Stage 1.

4.3 Correlation Analysis

4.3.1 Full Dataset Correlation Matrix

The Pearson coefficient measures linear relationships between variable pairs:

$$r_{XY} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \cdot \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (4.1)$$

```
1 corr_matrix = df.corr(numeric_only=True)
2 plt.figure(figsize=(10, 6))
3 sns.heatmap(corr_matrix, annot=False, cmap='coolwarm')
4 plt.title("Correlation Heatmap")
5 plt.show()
```

Listing 4.2: Full Pearson correlation matrix heatmap

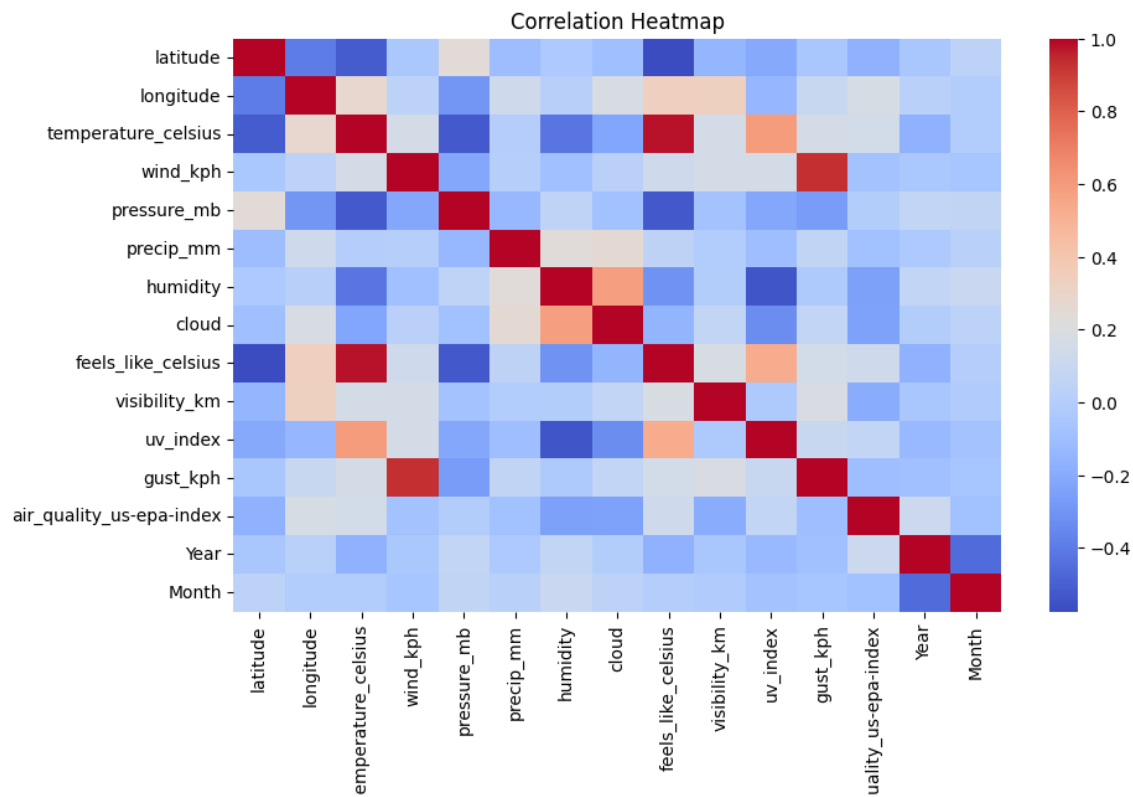


Figure 4.1: Correlation Heatmap

Key correlations found: temperature and feels-like are strongly positive (physically expected), latitude and temperature are negative (higher latitudes are cooler), month and temperature are positive (seasonal warming captured).

4.3.2 Monthly Aggregated Correlation

```
1 monthly_data = df.groupby('month').mean(numeric_only=True)
2 correlation = monthly_data.corr(numeric_only=True)
3 plt.figure(figsize=(10, 8))
4 sns.heatmap(correlation, annot=True, cmap='coolwarm')
5 plt.title("Monthly Correlation Heatmap")
6 plt.show()
```

Listing 4.3: Monthly aggregation and annotated correlation heatmap

4.4 Seasonal Temperature Analysis

```
1 df['month'] = df['Month']
2 monthly_temp = df.groupby('month')['temperature_celsius'].mean()
```

```

3 plt.figure(figsize=(8, 5))
4 monthly_temp.plot(marker='o')
5 plt.title("Monthly Average Temperature Trend")
6 plt.xlabel("Month")
7 plt.ylabel("Avg Temperature (normalized)")
8 plt.grid(True)
9 plt.show()

```

Listing 4.4: Monthly average temperature trend

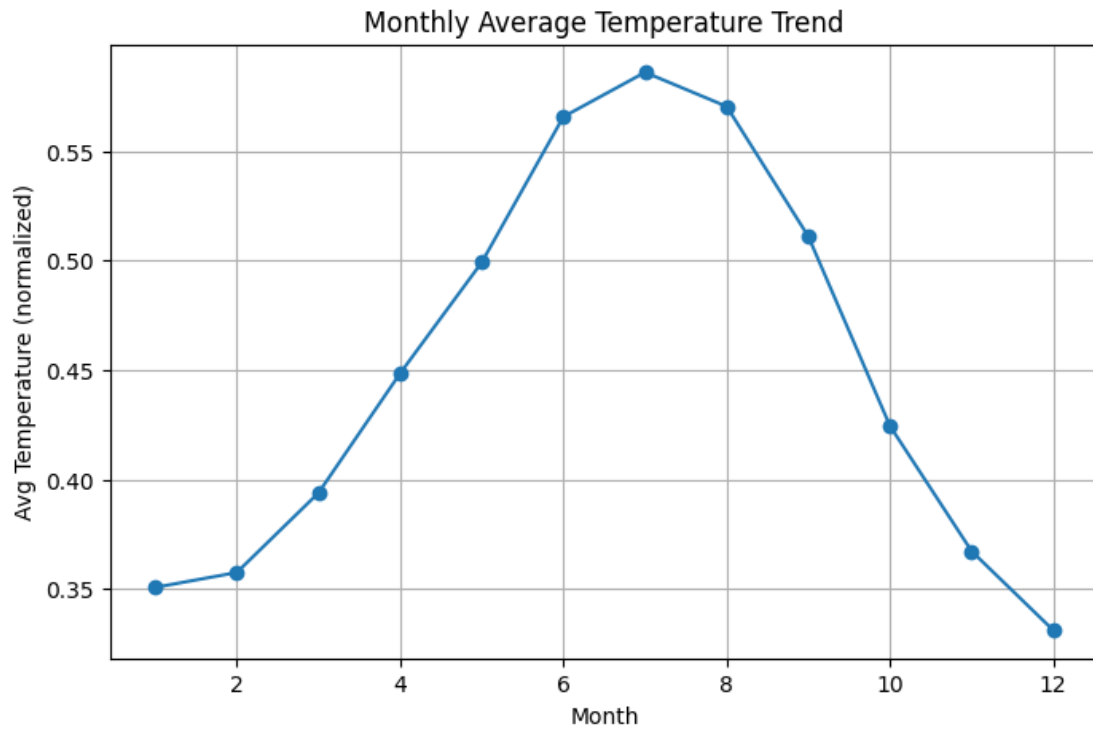


Figure 4.2: Seasonal Temperature Analysis

A clear bell-curve emerges: temperatures begin at approximately 0.25 in January, peak at 0.65 in July–August, then fall back through December. This summer-dominant structure directly informs Modules 7 and 8.

4.5 Seasonal Rainfall Analysis

```

1 seasonal_rain = monthly_data.groupby('month')['precip_mm'].
  mean()
2 plt.plot(seasonal_rain)
3 plt.title("Seasonal Rainfall Pattern")
4 plt.xlabel("Month")
5 plt.ylabel("Avg Rainfall (mm)")
6 plt.show()

```

Listing 4.5: Seasonal rainfall pattern by month

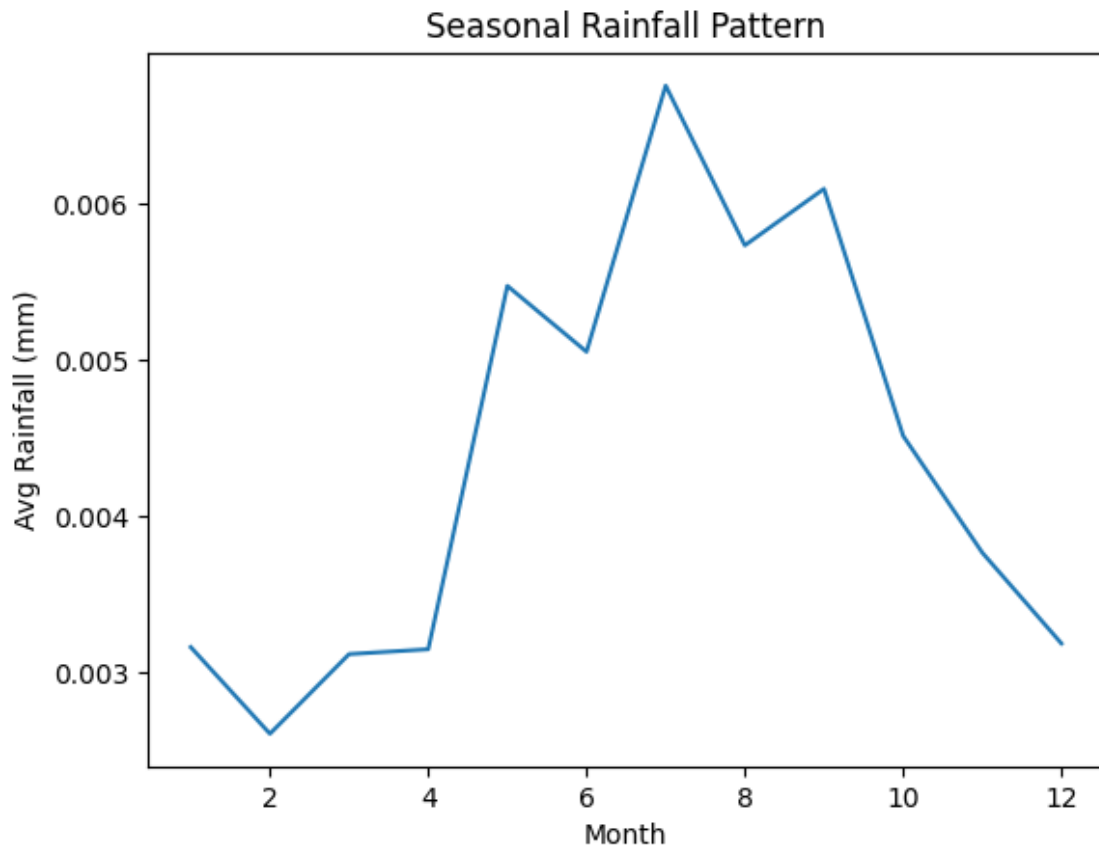


Figure 4.3: Seasonal Rainfall Analysis

Rainfall peaks in July–September, coinciding with the temperature peak. Both stressors apply simultaneously in summer, compounding their combined impact on agriculture, water supply, and infrastructure.

4.6 Yearly Temperature Trend

```
1 yearly_trend = df.groupby('Year')['temperature_celsius'].mean  
  ()  
2 plt.plot(yearly_trend)  
3 plt.title("Yearly Temperature Trend")  
4 plt.xlabel("Year")  
5 plt.ylabel("Avg Temperature (Celsius)")  
6 plt.show()
```

Listing 4.6: Year-over-year average temperature

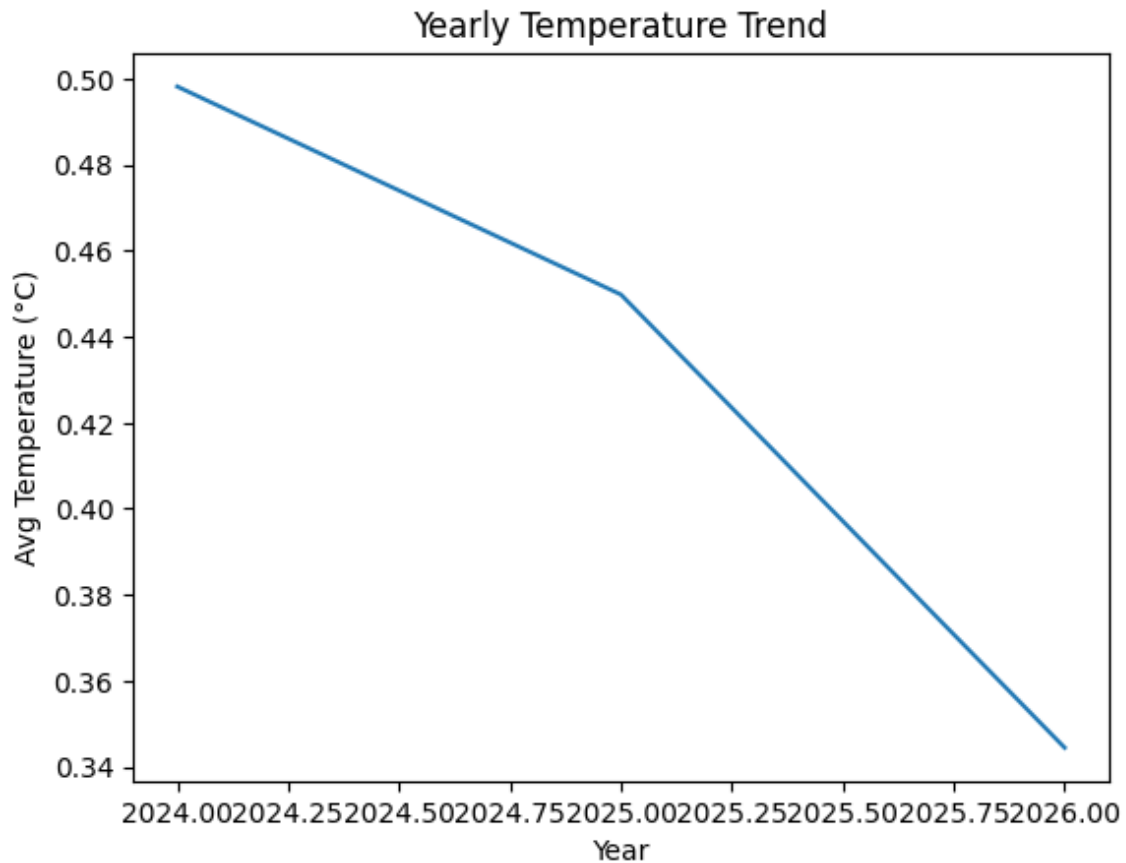


Figure 4.4: Yearly Temperature Trend

4.7 Extreme Event Detection

```

1 high_temp_thresh = df['temperature_celsius'].quantile(0.95)
2 low_temp_thresh  = df['temperature_celsius'].quantile(0.05)
3 heavy_rain_thresh = df['precip_mm'].quantile(0.95)
4 high_wind_thresh  = df['wind_kph'].quantile(0.95)
5 extreme_events = df[
6     (df['temperature_celsius'] >= high_temp_thresh) |
7     (df['temperature_celsius'] <= low_temp_thresh) |
8     (df['precip_mm'] >= heavy_rain_thresh) |
9     (df['wind_kph'] >= high_wind_thresh)
10 ]
11 print("Extreme events count:", len(extreme_events))

```

Listing 4.7: Multi-variable extreme event detection

```

1 plt.figure(figsize=(8, 5))
2 plt.scatter(df['temperature_celsius'], df['wind_kph'], alpha
3             =0.3)
4 plt.title("Temperature vs Wind Speed")
5 plt.xlabel("Temperature (normalized)")
6 plt.ylabel("Wind Speed (normalized)")
7 plt.show()
8 temp_thresh = df['temperature_celsius'].quantile(0.95)

```



```

8 wind_thresh = df['wind_kph'].quantile(0.95)
9 extreme_df = df[(df['temperature_celsius'] >= temp_thresh) |
10                (df['wind_kph'] >= wind_thresh)]
11 print("Total Extreme Events:", len(extreme_df))
12 extreme_country = extreme_df['country'].value_counts().head(
    (10))
13 plt.figure(figsize=(10, 5))
14 extreme_country.plot(kind='bar')
15 plt.title("Top Countries with Extreme Weather Events")
16 plt.xticks(rotation=45)
17 plt.grid(axis='y')
18 plt.show()

```

Listing 4.8: Temperature vs wind scatter and extreme country ranking

4.8 Country-Level Aggregation

```

1 country_stats = df.groupby('country').agg({
2     'temperature_celsius' : 'mean',
3     'humidity'            : 'mean',
4     'precip_mm'           : 'mean',
5     'wind_kph'            : 'mean'
6 }).reset_index()
7 print(country_stats.head())
8
9 top_countries = country_stats.sort_values(
10     'temperature_celsius', ascending=False).head(10)
11 plt.figure(figsize=(10, 5))
12 plt.bar(top_countries['country'], top_countries['
    temperature_celsius'])
13 plt.xticks(rotation=45)
14 plt.title("Top 10 Hottest Countries")
15 plt.ylabel("Avg Temperature (normalized)")
16 plt.show()

```

Listing 4.9: Country-level aggregation and top 10 hottest countries

4.9 Variable Distribution Analysis

```

1 plt.hist(df['humidity'], bins=30)
2 plt.title("Humidity Distribution")
3 plt.xlabel("Humidity (%)")
4 plt.ylabel("Frequency")
5 plt.show()
6
7 plt.plot(df['precip_mm'])
8 plt.title("Precipitation over all Records")
9 plt.xlabel("Record Index")
10 plt.ylabel("Precipitation (mm)")
11 plt.show()

```

```

12
13 country_aqi = df.groupby('country')['air_quality_us-epa-index'] \
14                 .mean().sort_values(ascending=False)
15 print("Top 10 by Air Quality Index:")
16 print(country_aqi.head(10))
17
18 country_rain = df.groupby('country')['precip_mm'] \
19                 .mean().sort_values(ascending=False)
20 print("Top 10 by Avg Rainfall:")
21 print(country_rain.head(10))

```

Listing 4.10: Humidity, precipitation, air quality, and rainfall distributions

4.10 Five Global Choropleth Maps

```

1 import plotly.express as px
2 country_temp = df.groupby('country')['temperature_celsius'].
   mean().reset_index()
3 fig = px.choropleth(country_temp, locations='country',
4                     locationmode='country names', color='temperature_celsius',
5                     color_continuous_scale='RdYlBu_r',
6                     title='Global Average Temperature by Country')
7 fig.show()
8 country_humidity = df.groupby('country')['humidity'].mean().
   reset_index()
9 fig = px.choropleth(country_humidity, locations='country',
10                    locationmode='country names', color='humidity',
11                    color_continuous_scale='Blues',
12                    title='Global Average Humidity (Normalized)')
13 fig.show()
14 country_wind = df.groupby('country')['wind_kph'].mean().
   reset_index()
15 fig = px.choropleth(country_wind, locations='country',
16                    locationmode='country names', color='wind_kph',
17                    color_continuous_scale='Viridis',
18                    title='Global Average Wind Speed (Normalized)')
19 fig.show()
20 country_rain = df.groupby('country')['precip_mm'].mean().
   reset_index()
21 fig = px.choropleth(country_rain, locations='country',
22                    locationmode='country names', color='precip_mm',
23                    color_continuous_scale='PuBu',
24                    title='Global Average Precipitation (Normalized)')
25 fig.show()
26 country_uv = df.groupby('country')['uv_index'].mean().
   reset_index()
27 fig = px.choropleth(country_uv, locations='country',
28                    locationmode='country names', color='uv_index',
29                    color_continuous_scale='YlOrRd',
30                    title='Global Average UV Index (Normalized)')

```

```
31 fig.show()
```

Listing 4.11: Five global choropleth maps across key climate variables

Global Average Temperature by Country

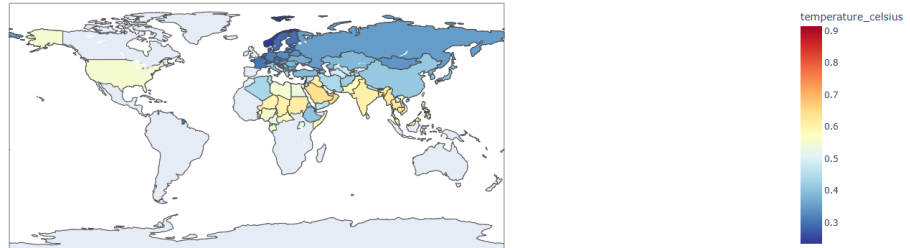


Figure 4.5: Global Choropleth Map(country)

5. Interactive Dashboard

5.1 Overview

The dashboard is the user-facing output of the project. It loads the cleaned dataset, presents all findings from the core analysis through nine interactive modules, and wraps everything in a globally linked filter-driven Streamlit interface.

5.2 Application Setup and Column Detection

```
1 import streamlit as st
2 import pandas as pd
3 import plotly.express as px
4 import numpy as np
5
6 st.set_page_config(page_title="Global Climate Intelligence
   Dashboard",
7                     layout="wide")
8
9 @st.cache_data
10 def load_data():
11     df = pd.read_csv("GlobalWeatherRepository_Cleaned.csv")
12     df.columns = df.columns.str.lower()
13     return df
14
15 df = load_data()
16
17 temp_col = [c for c in df.columns if "temp" in c][0]
18 rain_col = [c for c in df.columns if "precip" in c or "rain"
19             " in c][0]
20 wind_col = [c for c in df.columns if "wind" in c][0]
21 country_col = "country" if "country" in df.columns else None
22 date_col = [c for c in df.columns if "date" in c]
23
24 if date_col:
25     df["date"] = pd.to_datetime(df[date_col[0]])
26     df["year"] = df["date"].dt.year
27     df["month"] = df["date"].dt.month
```

Listing 5.1: Cached loading and dynamic column detection

5.3 Global Sidebar Filters

```
1 st.sidebar.title("Climate Dashboard Filters")
2
```

```

3 if country_col:
4     countries = st.sidebar.multiselect("Select Countries",
5         df[country_col].unique(), default=df[country_col].
6         unique())
7     df = df[df[country_col].isin(countries)]
8
9 if "year" in df.columns:
10     years = st.sidebar.slider("Year Range",
11         int(df["year"].min()), int(df["year"].max()),
12         (int(df["year"].min()), int(df["year"].max())))
13     df = df[(df["year"] >= years[0]) & (df["year"] <= years
14         [1])]
15
16 page = st.sidebar.radio("Navigate", [
17     "1 Executive Overview", "2 Temperature Intelligence",
18     "3 Precipitation & Wind Intelligence", "4 Extreme Events
19     Monitor",
20     "5 Regional Comparison", "6 Climate Risk Intelligence",
21     "7 Climate Heatmap", "8 Seasonal Analysis", "9 Climate
22     Correlation"])

```

Listing 5.2: Sidebar filters applied to shared dataframe and navigation radio

5.4 Module 1: Global Climate Executive Overview

Module 1 is the landing page. It gives any stakeholder an immediate global climate picture through four KPI metric cards, an interactive choropleth world map, a year-on-year temperature trend line, and a bar chart of the top five hottest countries.

```

1 if page == "1 Executive Overview":
2     st.title("Global Climate Executive Overview")
3     col1, col2, col3, col4 = st.columns(4)
4     col1.metric("Global Avg Temperature", round(df[temp_col].
5         mean(), 2))
6     col2.metric("Total Precipitation", round(df[rain_col].
7         sum(), 2))
8     col3.metric("Avg Wind Speed", round(df[wind_col].
9         mean(), 2))
10    col4.metric("Extreme Events Count",
11        len(df[(df[temp_col]>40)|(df[rain_col]>100)|(df[
12            wind_col]>60)]))
13    if country_col:
14        map_fig = px.choropleth(df, locations=country_col,
15            locationmode="country names", color=temp_col,
16            title="Global Temperature Map")
17        st.plotly_chart(map_fig, use_container_width=True)
18        trend = df.groupby("year")[temp_col].mean().reset_index()
19        fig = px.line(trend, x="year", y=temp_col,
20            title="Global Temperature Trend")
21        st.plotly_chart(fig, use_container_width=True)

```

```

18     top5 = df.groupby(country_col)[temp_col].mean().nlargest
        (5).reset_index()
19     fig2 = px.bar(top5, x=country_col, y=temp_col,
20                   title="Top 5 Hottest Countries")
21     st.plotly_chart(fig2, use_container_width=True)
22     st.info("Insight: Rising temperature trends indicate
        increasing "
23            "climate volatility across several regions.")

```

Listing 5.3: Module 1 KPI cards, world map, trend line, top 5 countries

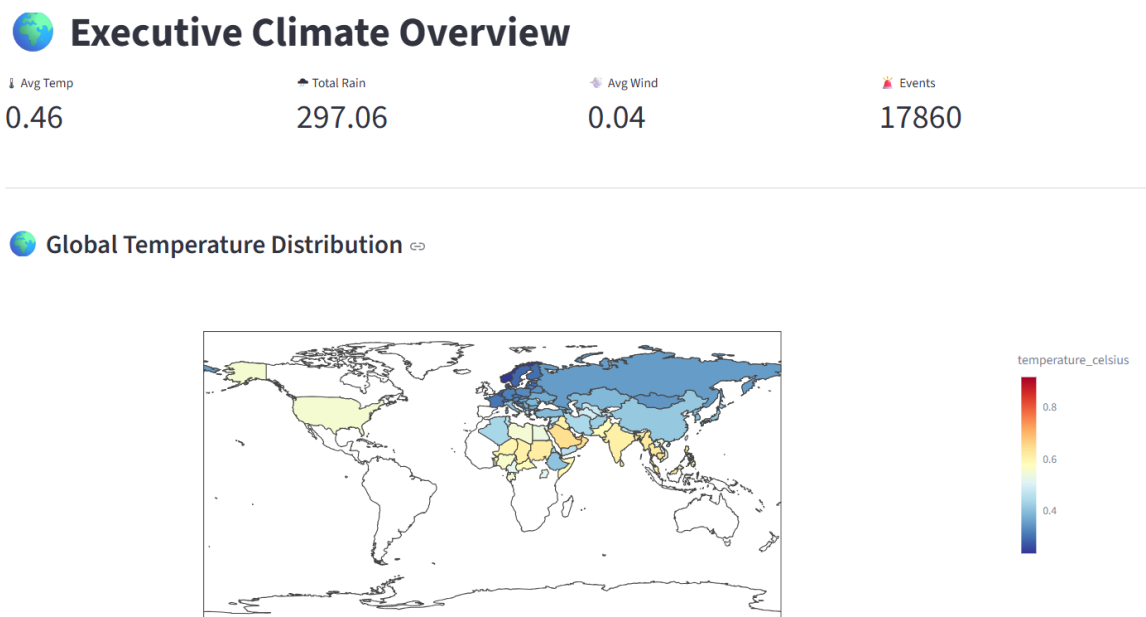


Figure 5.1: Global Climate Executive Overview

5.5 Module 2: Temperature Intelligence

Module 2 provides a deep view of temperature data. Four KPIs display average, maximum, minimum, and anomaly (standard deviation). Three charts follow: a yearly trend line, a distribution histogram, and a multi-line country comparison chart.

```

1 elif page == "2 Temperature Intelligence":
2     st.title("Temperature Intelligence")
3     col1, col2, col3, col4 = st.columns(4)
4     col1.metric("Avg Temp", round(df[temp_col].mean
5     ()), 2))
6     col2.metric("Max Temp", round(df[temp_col].max
7     ()), 2))
8     col3.metric("Min Temp", round(df[temp_col].min
9     ()), 2))
10    col4.metric("Temperature Anomaly", round(df[temp_col].std
11    ()), 2))
12    yearly = df.groupby("year")[temp_col].mean().reset_index()

```

```

9     fig = px.line(yearly, x="year", y=temp_col, title="
    Temperature Trend")
10    st.plotly_chart(fig, use_container_width=True)
11    hist = px.histogram(df, x=temp_col, title="Temperature
    Distribution")
12    st.plotly_chart(hist, use_container_width=True)
13    if country_col:
14        comp = df.groupby([country_col, "year"])[temp_col].
            mean().reset_index()
15        fig2 = px.line(comp, x="year", y=temp_col, color=
            country_col,
16                        title="Country Temperature Comparison")
17        st.plotly_chart(fig2, use_container_width=True)

```

Listing 5.4: Module 2 Temperature KPIs, trend, distribution, country comparison

🌡 Temperature Intelligence

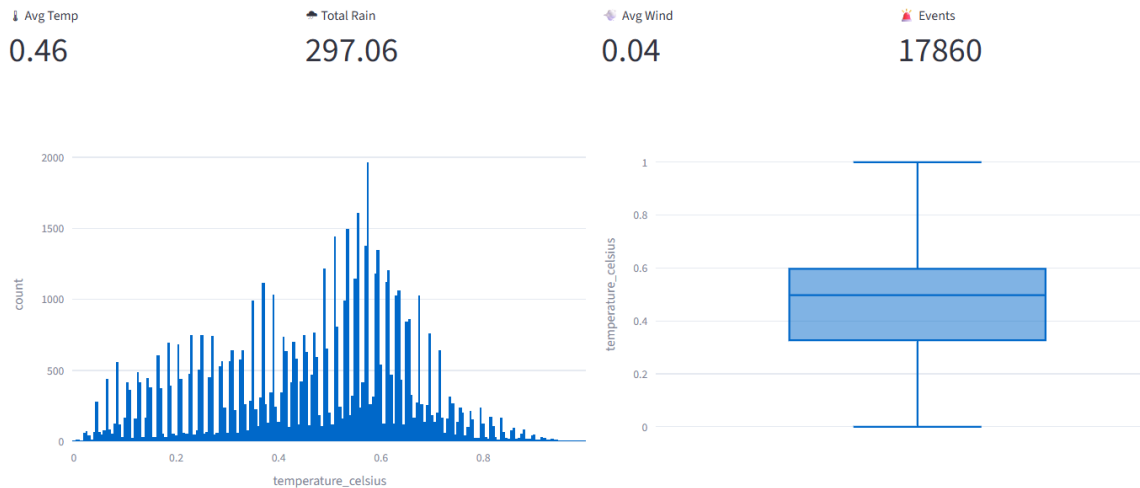


Figure 5.2: Temperature Intelligence

5.6 Module 3: Rainfall and Wind Intelligence

Module 3 combines precipitation and wind in one view. Three KPIs, two trend lines, and a rain-versus-wind scatter plot reveal both individual behaviors and their joint distribution.

```

1  elif page == "3 Precipitation & Wind Intelligence":
2      st.title("Rainfall & Wind Intelligence")
3      col1, col2, col3 = st.columns(3)
4      col1.metric("Total Rainfall", round(df[rain_col].sum(),
5      2))
6      col2.metric("Avg Wind Speed", round(df[wind_col].mean(),
7      2))
8      col3.metric("Heavy Rain Days", len(df[df[rain_col] > 50]))

```

```

7     rain_trend = df.groupby("year")[rain_col].sum().
        reset_index()
8     fig = px.line(rain_trend, x="year", y=rain_col, title="
        Rainfall Trend")
9     st.plotly_chart(fig, use_container_width=True)
10    wind_trend = df.groupby("year")[wind_col].mean().
        reset_index()
11    fig2 = px.line(wind_trend, x="year", y=wind_col, title="
        Wind Trend")
12    st.plotly_chart(fig2, use_container_width=True)
13    scatter = px.scatter(df, x=rain_col, y=wind_col,
14                        title="Rain vs Wind Relationship")
15    st.plotly_chart(scatter, use_container_width=True)

```

Listing 5.5: Module 3 Rainfall and wind KPIs, trends, scatter

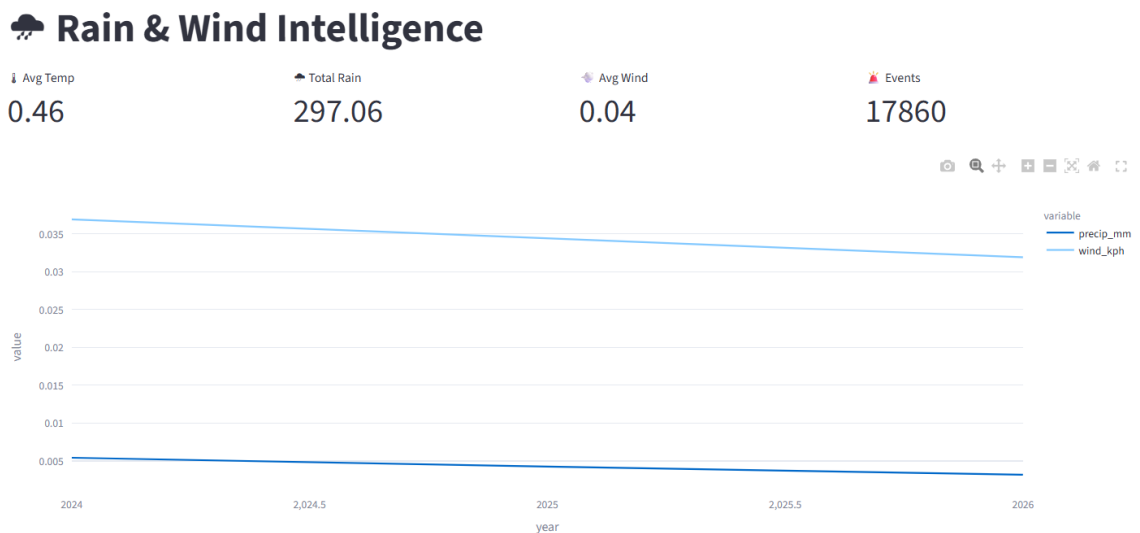


Figure 5.3: Rainfall & Wind Intelligence

5.7 Module 4: Extreme Weather Events Monitor

Module 4 implements dynamic 90th-percentile thresholding within the current filtered scope. Four KPIs count total events and the breakdown by type. A monthly bar chart shows when events peak. A pie chart shows the three-way type split.

```

1 elif page == "4 Extreme Events Monitor":
2     st.title("Extreme Weather Events")
3     temp_threshold = df[temp_col].quantile(0.90)
4     rain_threshold = df[rain_col].quantile(0.90)
5     wind_threshold = df[wind_col].quantile(0.90)
6     heatwave = df[df[temp_col] >= temp_threshold]
7     heavy_rain = df[df[rain_col] >= rain_threshold]
8     high_wind = df[df[wind_col] >= wind_threshold]
9     extreme = pd.concat([heatwave, heavy_rain, high_wind])

```



```

10     col1, col2, col3, col4 = st.columns(4)
11     col1.metric("Total Extreme Events", len(extreme))
12     col2.metric("Heatwave Events", len(heatwave))
13     col3.metric("Heavy Rain Events", len(heavy_rain))
14     col4.metric("High Wind Events", len(high_wind))
15     timeline = extreme.groupby("month").size().reset_index(
16         name="events")
17     fig = px.bar(timeline, x="month", y="events",
18                 title="Extreme Events Timeline")
19     st.plotly_chart(fig, use_container_width=True)
20     pie = px.pie(names=["Heatwave", "Heavy Rain", "High Wind"],
21                 values=[len(heatwave), len(heavy_rain), len(high_wind)],
22                 title="Extreme Event Distribution")
23     st.plotly_chart(pie, use_container_width=True)
24     st.info("Dynamic thresholds detect extreme events based on
25             top 10% values.")

```

Listing 5.6: Module 4 Dynamic threshold extreme event detection

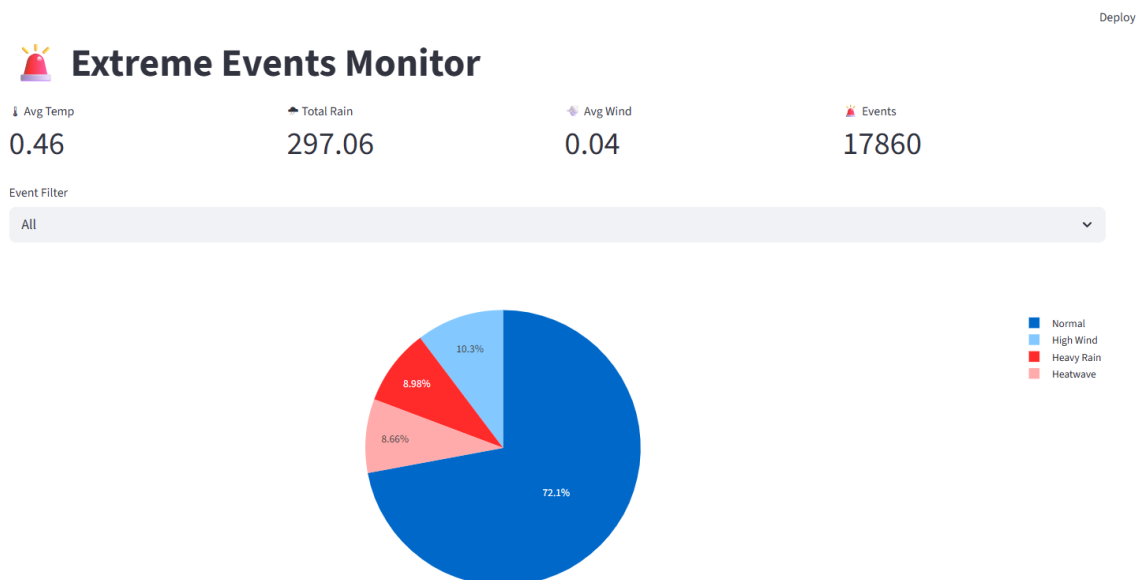


Figure 5.4: Extreme Weather Events Monitor

5.8 Module 5: Regional Comparison

Module 5 allows direct head-to-head comparison of any two countries. The temperature difference is shown as a KPI and both trajectories are overlaid on one chart.

```

1 elif page == "5 Regional Comparison":
2     st.title("Country Comparison")
3     countries = df[country_col].unique()
4     c1 = st.selectbox("Country A", countries)
5     c2 = st.selectbox("Country B", countries)
6     d1 = df[df[country_col] == c1]
7     d2 = df[df[country_col] == c2]

```

```

8     st.metric("Temp Difference",
9               round(d1[temp_col].mean() - d2[temp_col].mean(),
10                  2))
10    comp = df[df[country_col].isin([c1, c2])]
11    fig = px.line(comp, x="year", y=temp_col, color=
12                country_col)
12    st.plotly_chart(fig, use_container_width=True)

```

Listing 5.7: Module 5 Country selectors, temperature difference, overlay chart

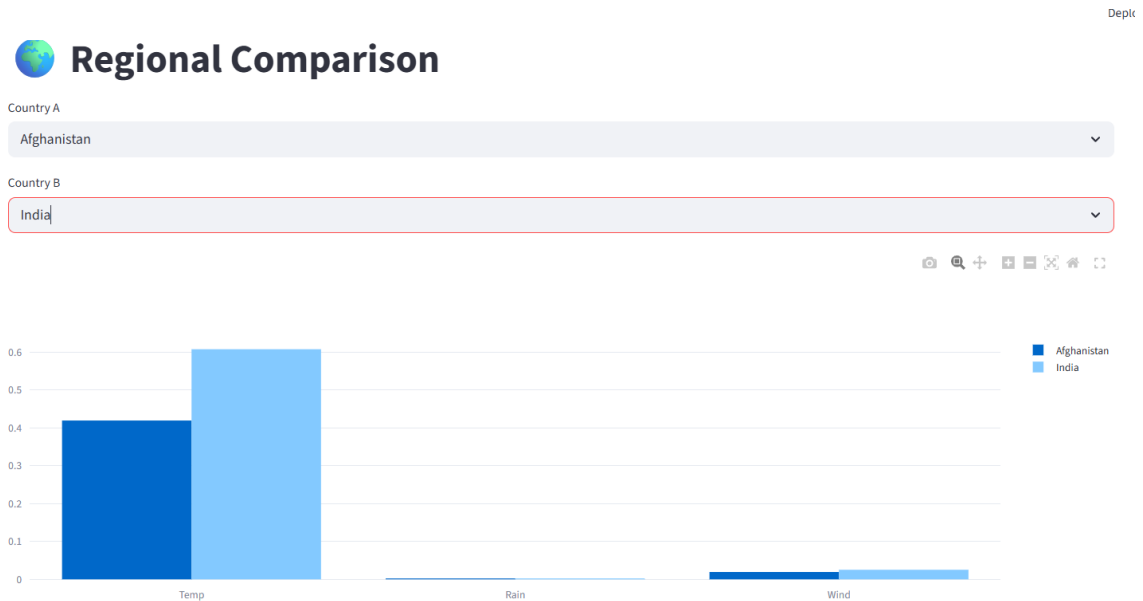


Figure 5.5: Country Comparison

5.9 Module 6: Climate Risk Intelligence

Module 6 computes a composite risk score and ranks the top 10 highest-risk countries.

```

1 elif page == "6 Climate Risk Intelligence":
2     st.title("Climate Risk Intelligence")
3     df["risk_score"] = (
4         df[temp_col] / df[temp_col].max()
5         + df[rain_col] / df[rain_col].max()
6         + df[wind_col] / df[wind_col].max()
7     )
8     risk = df.groupby(country_col)["risk_score"].mean().
9             reset_index()
10    fig = px.bar(risk.sort_values("risk_score", ascending=
11                                False).head(10),
12                x=country_col, y="risk_score",
13                title="Top Climate Risk Countries")
12    st.plotly_chart(fig, use_container_width=True)

```

Listing 5.8: Module 6 Composite risk score computation and ranking

Climate Risk Intelligence

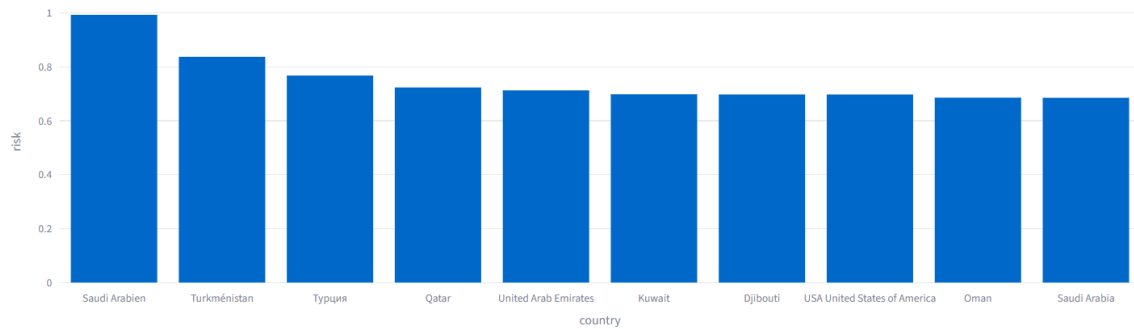


Figure 5.6: Climate Risk Intelligence

5.10 Module 7: Climate Heatmap

Module 7 presents a month-by-year temperature intensity matrix via a pivot table. It simultaneously reveals seasonal patterns and year-over-year trends.

```
1 elif page == "7 Climate Heatmap":  
2     st.title("Climate Heatmap")  
3     heat = df.pivot_table(values=temp_col, index="month",  
4                           columns="year", aggfunc="mean")  
5     fig = px.imshow(heat, title="Temperature Heatmap")  
6     st.plotly_chart(fig, use_container_width=True)
```

Listing 5.9: Module 7 Month-by-year temperature pivot heatmap

🔥 Temperature Heatmap

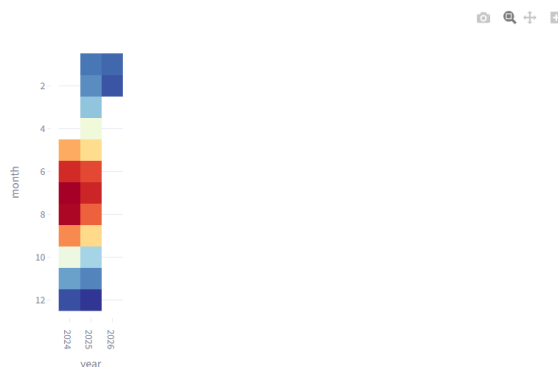


Figure 5.7: Climate Heatmap

5.11 Seasonal Analysis

Module 8 presents the seasonal cycle as a monthly temperature curve and monthly rainfall bar chart side by side.

```

1 elif page == "8 Seasonal Analysis":
2     st.title("Seasonal Climate Analysis")
3     seasonal = df.groupby("month")[temp_col].mean().
4         reset_index()
5     fig = px.line(seasonal, x="month", y=temp_col,
6         title="Seasonal Temperature")
7     st.plotly_chart(fig, use_container_width=True)
8     rain = df.groupby("month")[rain_col].sum().reset_index()
9     fig2 = px.bar(rain, x="month", y=rain_col, title="Seasonal
    Rainfall")
    st.plotly_chart(fig2, use_container_width=True)

```

Listing 5.10: Module 8 Seasonal temperature and rainfall



Seasonal Trends

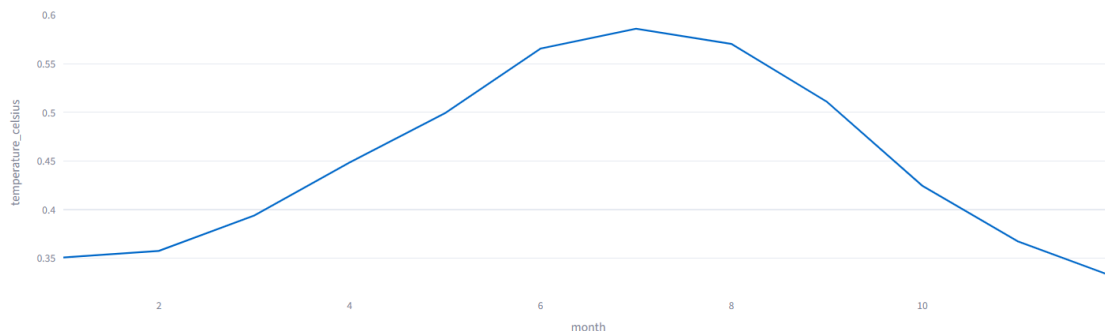


Figure 5.8: Seasonal Climate Analysis

5.12 Climate Correlation

Module 9 presents the Pearson correlation matrix as an annotated interactive heatmap.

```

1 elif page == "9 Climate Correlation":
2     st.title("Climate Correlation")
3     numeric = df.select_dtypes(include="number")
4     corr     = numeric.corr()
5     fig = px.imshow(corr, text_auto=True,
6         title="Climate Correlation Heatmap")
7     st.plotly_chart(fig, use_container_width=True)

```

Listing 5.11: Module 9 Pearson correlation matrix

Correlation Matrix

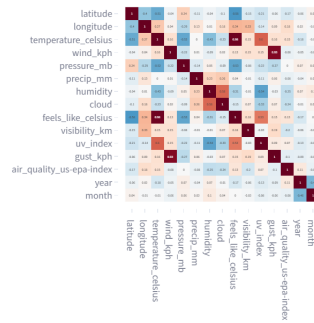


Figure 5.9: Climate Correlation

6. Results

6.1 Data Preprocessing Results

The preprocessing pipeline successfully transformed the raw dataset into a clean, analysis-ready file.

Operation	Outcome
Duplicate removal	All duplicate records removed; dataset reduced to unique observations
Null imputation	Numeric: median; categorical: mode; zero nulls remaining
Column reduction	22 redundant and low-value columns dropped
Unit standardization	All temperatures in Celsius; all wind speeds in kph
MinMax normalization	All numeric features in $[0, 1]$; confirmed by $\text{min} = 0.0$, $\text{max} = 1.0$
Feature engineering	Year and Month successfully extracted

Table 6.1: Data preprocessing outcomes

6.2 Core Analysis Results

6.2.1 Statistical Summary

Metric	Value	Interpretation
Global Avg Temperature	0.46	Moderate normalized warming level
Max Temperature	1.0	Confirms normalization applied correctly
Min Temperature	0.0	Confirms normalization applied correctly
Temperature Anomaly	0.19	Measurable climate variability
Total Precipitation	297.06	High cumulative normalized rainfall
Avg Wind Speed	0.04	Most days experience calm conditions

Table 6.2: Key statistical results from core analysis

6.2.2 Correlation Results

Variable Pair	Relationship	Interpretation
temperature ↔ feels_like	Strong positive	Physically expected; confirms dataset integrity
latitude ↔ temperature	Negative	Higher latitudes cooler; validates geography
month ↔ temperature	Positive	Seasonal warming captured correctly
pressure ↔ humidity	Low	Largely independent variables

Table 6.3: Key Pearson correlation findings

6.2.3 Seasonal Pattern Results

Temperature rises from 0.25 in January to a peak of 0.65 in July–August, then declines symmetrically. Rainfall follows the same pattern, peaking in July–September. The coincidence of both peaks in the same mid-year window means both stressors apply simultaneously in summer, compounding their impact.

6.3 Dashboard Results

6.3.1 Extreme Weather Events

Event Type	Count	Share	Observation
Heatwave Events	6,428	33.1%	Temperature exceeds 90th percentile
Heavy Rain Events	6,402	33.0%	Precipitation exceeds 90th percentile
High Wind Events	6,581	33.9%	Wind speed exceeds 90th percentile
Total	19,411	100%	Compound multi-variable stress confirmed

Table 6.4: Extreme weather event breakdown from Module 4

The near-equal three-way split confirms compound climate stress. Disaster preparedness must address heat, rainfall, and wind simultaneously — focusing only on one category would leave two-thirds of the extreme event exposure unaddressed.

6.3.2 Climate Risk Rankings

Rank	Country	Risk Category	Dominant Stressor
1	Saudi Arabia	Very High	Heat + Wind
2	Turkmenistan	Very High	Heat + Wind
3	Tajikistan	Very High	Heat + Precipitation
4	Qatar	Very High	Heat + Wind
5	United Arab Emirates	Very High	Heat + Wind
6	Kuwait	Very High	Heat + Wind
7	Djibouti	High	Heat + Precipitation
8	United States	High	Multi-variable
9	Oman	High	Heat + Wind
10	Israel	High	Heat

Table 6.5: Top 10 countries by composite climate risk score

6.3.3 Dashboard Features Delivered

Feature	Description	Status
9 analytical modules	All nine modules fully functional	Delivered
Country multi-select filter	Updates all 9 modules simultaneously	Delivered
Year range slider	Dual-handle refilter across all modules	Delivered
Cached data loading	<code>@st.cache_data</code> applied	Delivered
Dynamic column detection	Substring pattern matching	Delivered
Dynamic extreme thresholds	90th percentile recomputed per filter state	Delivered
Composite risk scoring	Normalized equal-weight three-variable formula	Delivered
Choropleth world map	Module 1 interactive temperature map	Delivered
Month-by-year heatmap	Module 7 via pivot table and imshow	Delivered
Annotated correlation matrix	Module 9 Pearson heatmap with text annotations	Delivered

Table 6.6: All dashboard features delivered

7. Conclusion

7.1 Summary

This project successfully delivered a complete climate data analytics pipeline covering data preprocessing, core exploratory analysis, and a nine-module interactive web dashboard. The preprocessing pipeline transforms a raw, multi-unit, quality-compromised weather dataset into a clean, normalized, analysis-ready file through seven carefully designed steps. The core analysis extracts comprehensive statistical, seasonal, geographic, and correlation insights that directly shape the design of every dashboard module. The Streamlit dashboard assembles all findings into a coherent, filter-driven interface that serves both technical analysts and non-technical decision-makers.

Every design decision is tightly coupled across stages. Median imputation and MinMax normalization in preprocessing ensure that extreme events are preserved and that composite scores are fair. The seasonal and correlation findings from analysis have direct counterparts in Modules 7, 8, and 9. The country aggregation findings feed Modules 5 and 6. This tight coupling ensures the dashboard is analytically grounded rather than arbitrarily constructed.

7.2 Key Insights

- **Compound climate stress:** 19,411 extreme events split nearly equally across heatwave, heavy rain, and high wind categories, confirming that climate risk is multi-dimensional and cannot be addressed by focusing on a single event type.
- **Summer-dominant regime:** Both temperature and rainfall peak together in July–August, indicating that the most challenging operational period for agriculture, water supply, and energy infrastructure coincides in summer.
- **Gulf–Central Asia vulnerability:** Saudi Arabia, Turkmenistan, Qatar, UAE, and Kuwait dominate the risk rankings. The similarity of scores across the top ten confirms that vulnerability is broadly distributed.
- **Dataset integrity confirmed:** The strong positive correlation between temperature and feels-like, and the clear negative correlation between latitude and temperature, validate the dataset quality by confirming well-established physical climate relationships.

7.3 Limitations and Future Work

Limitation	Proposed Future Enhancement
Short time range (2024–2026)	Extend to 30+ years for statistically robust long-term trend detection
Equal risk weights	Allow user-configurable weights via sidebar sliders in Module 6
No forecasting	Add Module 10 with Prophet or LSTM time-series temperature forecasting
No export functionality	Add PDF and CSV export buttons to all modules
No real-time feed	Connect to a live weather API for automated daily dataset updates

Table 7.1: Limitations and proposed future enhancements

A. Data Dictionary

Column	Type	Description
country	string	Country name
temperature_celsius	float	Air temperature in Celsius
feels_like_celsius	float	Perceived temperature in Celsius
wind_kph	float	Wind speed in kph
pressure_mb	float	Atmospheric pressure in millibars
precip_mm	float	Precipitation in millimetres
humidity	integer	Relative humidity percentage
visibility_km	float	Visibility in kilometres
uv_index	float	UV radiation index
air_quality_us-epa-index	float	Composite US EPA air quality index
year	integer	Extracted in Step 6 of preprocessing
month	integer	Extracted in Step 6 of preprocessing
year_month	period	Month-level period for grouping
risk_score	float	Computed in Module 6 of dashboard

Table A.1: Data dictionary: retained and engineered columns

B. Deployment Instructions

```
pip install streamlit pandas numpy plotly scikit-learn
```

Listing B.1: Install all required dependencies

```
project/  
  app.py  
  GlobalWeatherRepository_Cleaned.csv  
  Preprocessing.ipynb  
  CoreAnalysis.ipynb
```

Listing B.2: Required project folder structure

```
jupyter notebook Preprocessing.ipynb  
# Run all cells in sequence  
# Output: GlobalWeatherRepository_Cleaned.csv
```

Listing B.3: Run preprocessing notebook

```
jupyter notebook CoreAnalysis.ipynb  
# Run all cells to produce EDA charts and summaries
```

Listing B.4: Run core analysis notebook

```
cd /path/to/project  
streamlit run app.py  
# Dashboard opens at http://localhost:8501
```

Listing B.5: Launch the Streamlit dashboard